

one asset, one lien.

A neutral first to file lien registry on Creditcoin. It witnesses pledges from lending protocols that never integrated with it, and refuses the second claim on the same collateral.

BUIDL CTC 2026 · RWA

singleton.unitynodes.com

0x8170B29e · CC3 testnet, verified

A borrower pledges one deed twice, and nobody notices.

01

Harbor lends

1,000 against a tokenised deed. Non custodial, so the borrower keeps the token.

02

Meridian lends

750 against the same deed, an hour later. A different contract, no shared code or storage.

03

Both are live

Neither protocol can see the other's record. The collateral is now promised twice.

Custody prevents this mechanically. Everywhere custody is absent, it is live and unaddressed.

A contract cannot read a log. Not even its own.

LOG0 through LOG4 are write only. Receipts live in a trie execution never touches. BLOCKHASH reaches back 256 blocks.

So a neutral witness of somebody else's lending is either an off chain indexer, which is a trusted party and therefore not neutral, or a contract that consumes an inclusion proof of that log.

The second exists on Creditcoin and nowhere else.

BlockProver 0x0FD2 re-checks a merkle and continuity proof synchronously, inside the transaction that accepts the pledge.

This is a property of the machine, not a positioning claim.

Three steps, nobody trusted for any of them.

01 **The protocol's own log**

A lender emits its native event. NFTfi and Blur Blend are read on Ethereum mainnet, unmodified and unaware. Nothing is deployed there and nothing is asked of them.

02 **BlockProver 0x0FD2**

The precompile verifies the inclusion proof inside the accepting transaction, past a stated confirmation depth read from ChainInfo 0x0fD3. The registry checks the source receipt status itself, because the precompile does not.

03 **The register**

First to file is recorded and given a soulbound certificate. Anything second reverts. A nullifier per operation makes each proof spendable once.

The refusal is a failed transaction anybody can open.

testnet

Creditcoin

Blockchain

Tokens

Charts & stats

API

Other

Search by address / txn hash / block / token/...

Log in

Transaction details

⚡ 0x59...8516

failed to call

registerPledge

on

SingletonRegistry

Details

Token transfers

Internal txns

Logs

State

Raw trace

This is a testnet transaction only

Transaction hash

0xe575aaf998bd2baaaee89629c7f9c2bab4faf5bc2eebd207cc0ee4a680e859f9

Status and method

Failed

registerPledge

Hide reason

Method id

6b20dfe0

Call

AssetNotFree(bytes32 assetKey, address incumbent)

Name	Type	Data
assetKey	bytes32	0x8927ac9951b14386e6acf9c4865f37908223c007edc4d987aec68f7ad9334171

Blockscout, CC3 testnet. The registry is verified there, so the custom error decodes into the asset key and the address of the lender who filed first.

Half the proofs read protocols that never heard of us.

NFTfi v3, loan 16928

Taken at mainnet block 25,506,517, repaid at 25,717,460. Real borrower, real NFT, 0.07 WETH.

Blur Blend, two liens

One closed by repayment, one seized after a failed auction. A lien ends in more than one way and an adapter that knew only repayment would leave seized liens on file forever.

Creditcoin attests Ethereum, so an existing loan can simply be read. No mainnet deployment, no funds there, no cooperation from either protocol.

That removes the staged demo objection: the two Sepolia lenders are ours, and six of the twelve proofs are not.

Depth, stated as calls rather than as adjectives.

In use

BlockProver 0x0FD2, view form, in the accepting transaction.
ChainInfo 0x0FD3 for the attested tip and for resolving a chain key from a chain id. The deployed EvmV1Decoder. @gLuwa/usc-sdk for proof assembly.

Both

attested source chains: `get_supported_chains` reports two on CC3 testnet and both are read. Six proofs each.

Refused, with reasons

SubstrateTransfer moves CTC and a register holds none. The signature verifiers want Substrate keys and the evidence here is an inclusion proof. Staking and the legacy loan pallets are not reachable from Solidity at all. Writability is not shipped.

Not built, and named

Batch verification, the ChainInfo bounds queries, and AttestorStash. The full list with reasons is in `docs/SURFACES.md`.

Two independent reviews in one day. Both found real bugs.

01 **Receipt poisoning**

A borrower could attach a decoy log carrying the registry's own event signature and make their genuine pledge permanently unregistrable.

02 **The same mistake, one door along**

The fix scoped log counting to the chosen emitter, but the emitter was still inferred from the receipt, which the borrower orders. One log from any other allowlisted protocol suppressed the pledge again.

03 **What changed**

The relayer now names the emitter and the log index. The registry searches for nothing. A batch pledge became supported rather than refused, and mainnet gas fell from 716k to 634k.

Also corrected: a caveat claiming an administrator cannot fabricate, which was false, and the vacuous test that backed it. 61 tests, every attack kept as a regression.

Said before anybody has to ask.

A positive record

An asset the register calls free is one nobody registered here, not one nobody pledged. Attestcoin proves a transaction happened, never that one did not.

The adapter is trusted

The proof decides whether a log exists; the adapter decides what it means. Bounded interpretation of a proven fact, in eighty lines of pure code, not unbounded trust in existence.

The key is free here only

`keccak(chainKey, token, tokenId)` names an on chain asset with no agreement between protocols. Off chain invoices are the larger market and the harder key.

UCC-9 has governed a trillion dollar lien market on exactly this basis for fifty years: prevention comes from priority and from the habit of checking, not from omniscience.

Running, and checkable without asking us for anything.

12

inclusion proofs on CC3 testnet,
every hash published

2 / 2

attested source chains read, Sepolia
and Ethereum mainnet

61

tests, including both reviews kept as
regressions

0

wallets, backends and indexers
between the page and the chain

`singleton.unitynodes.com`

`/demo, 1:25, captioned`

`github.com/UnityNodes/singleton`

`0x8170B29ee13Ee120952F44eb17F00e0B161dfde7`